

# **Tối ưu hóa thời gian khởi động lạnh (Cold-start) cho Serverless Knative sử dụng Lập lịch Dự đoán và Học Máy**

**Nguyễn Ngọc Tiểu Long (250202047)**

*Khoa Công nghệ Thông tin*

*Đại học Công nghệ Thông tin Thành phố Hồ Chí Minh*

**Giảng viên hướng dẫn: TS. Huỳnh Văn Đặng**

*Khoa Công nghệ Thông tin*

*Đại học Công nghệ Thông tin Thành phố Hồ Chí Minh*

Tháng 7 năm 2026

## Abstract

Điện toán phi máy chủ (Serverless computing) đang ngày càng phổ biến nhờ khả năng tự động mở rộng quy mô và mô hình thanh toán theo mức sử dụng thực tế (pay-as-you-go). Tuy nhiên, hiện tượng "khởi động lạnh" (cold-start) vẫn là một thách thức lớn, gây ra độ trễ đáng kể cho các yêu cầu ban đầu khi không có phiên bản (instance) nào đang hoạt động. Nghiên cứu này giải quyết bài toán khởi động lạnh trong Knative — một nền tảng serverless dựa trên Kubernetes — bằng cách đề xuất phương pháp lập lịch dự đoán sử dụng Học máy (Machine Learning). Thay vì phụ thuộc vào cơ chế mở rộng phản ứng mặc định của Knative (Knative Pod Autoscaler), chúng tôi đã phát triển một mô hình dự báo chuỗi thời gian Bộ nhớ ngắn hạn dài (Long Short-Term Memory - LSTM) để dự đoán xu hướng tải công việc trong tương lai dựa trên dữ liệu lưu lượng truy cập lịch sử được thu thập qua Prometheus. Dựa trên các dự đoán này, một chiến lược "Làm ấm trước" (Pre-warming) chủ động được triển khai để khởi tạo và giữ cho một số lượng pod thích hợp ở trạng thái sẵn sàng trước khi lưu lượng truy cập thực sự tăng vọt. Kết quả thực nghiệm trong môi trường mô phỏng chứng minh rằng hệ thống dự đoán đề xuất của chúng tôi giảm thiểu đáng kể độ trễ khởi động lạnh so với cơ chế mặc định. Hơn nữa, nghiên cứu đánh giá sự cân bằng giữa việc giảm độ trễ và lượng tài nguyên nhân rồi tiêu thụ, chứng minh rằng giải pháp này cung cấp một cách tiếp cận hiệu quả và cân bằng để tối ưu hóa hiệu suất serverless trong các môi trường lưu lượng truy cập biến động.

## Lời cảm ơn

Trước tiên, tôi xin gửi lời cảm ơn sâu sắc nhất đến giảng viên hướng dẫn của mình, Tiến sĩ Huỳnh Văn Đăng, vì sự chỉ bảo tận tình và những định hướng quý báu của thầy trong suốt quá trình thực hiện đồ án này. Kiến thức chuyên môn và những phản hồi sâu sắc từ thầy chính là yếu tố then chốt giúp tôi vượt qua những khó khăn khi tiếp cận với các công nghệ mới như Kubernetes, Knative, Học máy và hoàn thành tốt đề tài nghiên cứu.

Bên cạnh đó, tôi cũng xin gửi lời tri ân chân thành đến quý thầy cô tại Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Thông tin - ĐHQG-HCM vì đã trang bị cho tôi một nền tảng kiến thức vững chắc và một môi trường học tập tuyệt vời trong suốt thời gian qua.

# Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Tổng quan</b>                                                        | <b>7</b>  |
| 1.1      | Giới thiệu chung về Điện toán Đám mây và Serverless Computing . . . . . | 7         |
| 1.2      | Bài toán Khởi động lạnh (Cold-start Problem) . . . . .                  | 7         |
| 1.3      | Nền tảng Knative và Hạn chế của Cơ chế mở rộng phản ứng . . . . .       | 8         |
| 1.4      | Giải pháp Đề xuất: Lập lịch Dự đoán (Predictive Scheduling) với Học máy | 8         |
| 1.5      | Mục tiêu và Phạm vi nghiên cứu . . . . .                                | 8         |
| 1.6      | Cấu trúc của Luận văn . . . . .                                         | 9         |
| <b>2</b> | <b>Cơ sở Lý thuyết và Công nghệ</b>                                     | <b>10</b> |
| 2.1      | Hệ sinh thái Kubernetes . . . . .                                       | 10        |
| 2.2      | Nền tảng Knative . . . . .                                              | 10        |
| 2.2.1    | Knative Serving . . . . .                                               | 10        |
| 2.2.2    | Knative Pod Autoscaler (KPA) và Activator . . . . .                     | 11        |
| 2.3      | Hệ thống Giám sát Prometheus . . . . .                                  | 12        |
| 2.4      | Bài toán Dự báo Chuỗi thời gian (Time-series Forecasting) . . . . .     | 13        |
| 2.5      | Mạng nơ-ron Bộ nhớ Ngắn-Dài hạn (Long Short-Term Memory - LSTM) .       | 14        |
| 2.5.1    | Cổng Quên (Forget Gate) . . . . .                                       | 14        |
| 2.5.2    | Cổng Đầu vào (Input Gate) . . . . .                                     | 15        |
| 2.5.3    | Cổng Đầu ra (Output Gate) . . . . .                                     | 15        |
| <b>3</b> | <b>Kiến trúc Đề xuất và Hiện thực</b>                                   | <b>16</b> |
| 3.1      | Tổng quan Kiến trúc Hệ thống Predictive Scaler . . . . .                | 16        |
| 3.2      | Hiện thực Dịch vụ Dự báo Học máy (ML Service) . . . . .                 | 16        |
| 3.2.1    | Cơ chế giả lập đỉnh tải 3 pha (3-Phase Spike Simulation) . . . . .      | 17        |
| 3.3      | Hiện thực Bộ điều khiển Tỷ lệ (Predictive Controller) . . . . .         | 18        |
| 3.3.1    | Thuật toán làm mịn và Cửa sổ ổn định (Smoothing Algorithm) . .          | 19        |
| 3.4      | Hiện thực Bảng điều khiển Giám sát (Executive Dashboard) . . . . .      | 19        |
| <b>4</b> | <b>Đánh giá và Kết quả Thực nghiệm</b>                                  | <b>21</b> |
| 4.1      | Thiết lập Môi trường Thực nghiệm (Experimental Setup) . . . . .         | 21        |
| 4.2      | Các Chỉ số Đánh giá (Evaluation Metrics) . . . . .                      | 21        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.3      | Kết quả Thực nghiệm Mô phỏng Đỉnh tải . . . . .                   | 22        |
| 4.3.1    | Phân tích Quỹ đạo Mở rộng (Scaling Trajectory) . . . . .          | 22        |
| 4.3.2    | Đánh giá biểu đồ Vòng đời (Lifecycle State - Pie Chart) . . . . . | 23        |
| 4.4      | Thảo luận và Sự đánh đổi (Trade-offs) . . . . .                   | 24        |
| <b>5</b> | <b>Kết luận và Hướng phát triển</b>                               | <b>24</b> |
| 5.1      | Kết luận . . . . .                                                | 24        |
| 5.2      | Hướng phát triển tương lai . . . . .                              | 25        |

## List of Figures

|   |                                                                                                             |    |
|---|-------------------------------------------------------------------------------------------------------------|----|
| 1 | Cơ chế xử lý Khởi động lạnh (Cold-start) thông qua Activator trong Knative                                  | 12 |
| 2 | Sơ đồ kiến trúc bên trong của một khối Tế bào LSTM . . . . .                                                | 14 |
| 3 | Sơ đồ luồng hoạt động của hệ thống Predictive Scaler . . . . .                                              | 17 |
| 4 | Đồ thị mô phỏng tấn công mạng 3 pha (3-Phase Spike Simulation) . . . .                                      | 18 |
| 5 | Lưu đồ thuật toán Làm mịn (Smoothing Algorithm) điều tiết quá trình<br>Mở rộng/Thu hẹp . . . . .            | 20 |
| 6 | Giao diện Giám sát Executive Dashboard hiển thị Quỹ đạo mở rộng (Scaling Trajectory) và Pie Chart . . . . . | 22 |
| 7 | Phân tích chuyên sâu Mô hình LSTM (ML Model Insights) và Đường dự<br>báo tương lai . . . . .                | 23 |

## List of Tables

|   |                                                                                                  |    |
|---|--------------------------------------------------------------------------------------------------|----|
| 1 | So sánh đặc tính giữa Knative Pod Autoscaler (KPA) và Kubernetes HPA                             | 11 |
| 2 | Tổng hợp kết quả so sánh hiệu năng giữa Knative mặc định và Lập lịch<br>Dự đoán (LSTM) . . . . . | 22 |

# 1 Tổng quan

## 1.1 Giới thiệu chung về Điện toán Đám mây và Serverless Computing

Gần đây, kiến trúc nguyên khối (Monolithic) đang dần nhường chỗ cho hệ thống phân tán và vi dịch vụ (Microservices). Điện toán đám mây (Cloud Computing) phát triển nhanh chóng, kéo theo nhiều cách thức mới để triển khai ứng dụng. Trong số đó, mô hình Điện toán phi máy chủ (Serverless Computing) đang nhận được nhiều sự chú ý [5, 3].

Với Serverless, lập trình viên không còn phải quản lý hay duy trì các máy chủ vật lý hay máy ảo nữa. Họ chỉ cần viết code. Hệ thống sẽ tính tiền dựa trên lượng tài nguyên dùng thực tế (pay-as-you-go). Đặc biệt nhất là tính năng "Scale-to-Zero" – hệ thống có thể tắt toàn bộ các ứng dụng (instances) đang chạy nếu không có request nào, giúp tiết kiệm chi phí đáng kể. Mặc dù rất tiện lợi, nhưng mô hình này lại sinh ra một vấn đề kỹ thuật lớn: lỗi khởi động lạnh.

## 1.2 Bài toán Khởi động lạnh (Cold-start Problem)

Khởi động lạnh (Cold-start) diễn ra khi có một request gửi đến ứng dụng Serverless, nhưng lúc đó ứng dụng lại đang ngủ (nghĩa là số lượng pod bằng 0). Khi đó, hệ thống đám mây sẽ phải làm khá nhiều việc để xử lý request này:

1. Xin cấp tài nguyên (CPU, Memory).
2. Kéo (pull) image từ kho (Registry).
3. Cấu hình mạng (cấp phát IP).
4. Chạy ứng dụng bên trong container. Tùy framework mà bước này nhanh hay chậm, như Java thì phải bật JVM, còn Python thì load thư viện.

Tùy vào độ phức tạp của ứng dụng, toàn bộ quá trình này mất khoảng vài giây, có khi lên đến cả chục giây. Hậu quả là request đầu tiên của người dùng sẽ bị nghẽn (block) chờ cho đến khi ứng dụng chạy xong. Đối với những hệ thống đòi hỏi thời gian phản hồi siêu thấp (như giao dịch tài chính hay thời gian thực), độ trễ này ảnh hưởng rất xấu đến trải nghiệm người dùng và dễ gây vi phạm cam kết dịch vụ (SLA).

### 1.3 Nền tảng Knative và Hạn chế của Cơ chế mở rộng phản ứng

Knative là một công cụ mã nguồn mở được chạy trên nền Kubernetes, giúp mang lại tính năng Serverless cho các cụm container [1]. Cơ chế mở rộng số lượng pod mặc định của Knative là Knative Pod Autoscaler (KPA). Nó hoạt động theo kiểu phản ứng (Reactive Autoscaling).

Nói đơn giản là KPA sẽ đợi đến khi thấy lượng truy cập thực tế vượt mức quy định thì mới bắt đầu tăng số pod lên. Mặc dù cách này khá an toàn, nhưng vì tính chất "thấy request mới scale", hệ thống chắc chắn sẽ bị dính khởi động lạnh nếu bỗng nhiên lượng truy cập tăng vọt từ mức 0.

### 1.4 Giải pháp Đề xuất: Lập lịch Dự đoán (Predictive Scheduling) với Học máy

Thay vì thụ động đợi tải tăng, nghiên cứu này xây dựng một giải pháp Lập lịch dự đoán (Predictive Scheduling) chủ động [2]. Mục tiêu là dự đoán trước lượng truy cập để tạo sẵn pod (Pre-warming). Như vậy khi request tới thì pod đã có sẵn rồi, không còn tình trạng khởi động lạnh nữa.

Để làm được việc này, chúng tôi áp dụng mô hình mạng nơ-ron hồi quy dài-ngắn hạn (LSTM) [4]. LSTM xử lý rất tốt các chuỗi dữ liệu theo thời gian (Time-series data). Dữ liệu về số lượng request (RPS) sẽ được gom lại qua Prometheus, sau đó nạp vào mô hình LSTM để dự báo tải. Dựa trên số liệu dự báo, một chương trình tự viết (Custom Controller) sẽ điều khiển Kubernetes tăng hoặc giảm số pod cho phù hợp.

### 1.5 Mục tiêu và Phạm vi nghiên cứu

#### Mục tiêu chính:

- Phân tích và làm rõ cơ chế hoạt động, cũng như các điểm nghẽn gây ra hiện tượng khởi động lạnh trên nền tảng Knative.
- Thiết kế, phát triển và huấn luyện mô hình Học sâu LSTM có khả năng dự đoán biến động lưu lượng mạng với độ chính xác cao.
- Phát triển một Kubernetes Custom Controller bằng ngôn ngữ Python có khả năng



tương tác với API của Kubernetes để thao tác mở rộng quy mô dựa trên kết quả dự đoán.

- Đánh giá hiệu suất của giải pháp đề xuất thông qua các mô phỏng (simulations) về lưu lượng truy cập tăng vọt, so sánh với cơ chế Autoscaler mặc định của Knative.

**Phạm vi nghiên cứu:** Nghiên cứu này tập trung vào việc tối ưu hóa độ trễ ở cấp độ nền tảng (Platform level) cho các cụm Kubernetes/Knative, không đi sâu vào việc tối ưu hóa mã nguồn (code-level) của từng ngôn ngữ lập trình cụ thể (ví dụ như biên dịch AOT của Java hay tối ưu hóa thư viện Python). Khung đánh giá được giới hạn trong một môi trường mô phỏng cục bộ (Local Kubernetes Cluster) sử dụng các mẫu tải nhân tạo (synthetic workloads) được thiết kế để mô phỏng các đợt tấn công DDoS hoặc sự kiện Flash-sale.

## 1.6 Cấu trúc của Luận văn

Nội dung của luận văn được tổ chức thành 5 chương chính như sau:

- **Chương 1 - Tổng quan:** Giới thiệu bối cảnh, bài toán, giải pháp đề xuất và mục tiêu nghiên cứu.
- **Chương 2 - Cơ sở lý thuyết và Các công nghệ liên quan:** Trình bày chi tiết về kiến trúc Kubernetes, Knative, KPA, và lý thuyết nền tảng về mô hình dự báo chuỗi thời gian LSTM.
- **Chương 3 - Kiến trúc Đề xuất và Hiện thực:** Đi sâu vào thiết kế hệ thống, giải thuật làm mịn (Smoothing Algorithm), cơ chế giả lập tải 3 pha (3-Phase Spike Simulation) và giao diện giám sát (Executive Dashboard).
- **Chương 4 - Đánh giá và Kết quả Thực nghiệm:** Phân tích các số liệu thu thập được từ thực nghiệm, so sánh hiệu năng và đánh giá trực quan qua Dashboard.
- **Chương 5 - Kết luận và Hướng phát triển:** Tóm tắt lại những đóng góp của đề tài và mở ra các định hướng cải tiến trong tương lai.

## 2 Cơ sở Lý thuyết và Công nghệ

### 2.1 Hệ sinh thái Kubernetes

Kubernetes là một nền tảng mã nguồn mở mang tính cách mạng trong việc tự động hóa triển khai, mở rộng quy mô và quản lý các ứng dụng được đóng gói trong container (containerized applications). Kubernetes giải quyết bài toán phức tạp của việc phân phối tài nguyên trên một cụm (cluster) gồm nhiều máy chủ vật lý hoặc máy ảo, đảm bảo tính sẵn sàng cao (High Availability) và khả năng chịu lỗi (Fault Tolerance).

Một trong những tính năng cốt lõi của Kubernetes là **Horizontal Pod Autoscaler (HPA)**. HPA tự động mở rộng hoặc thu hẹp số lượng bản sao (replicas) của một Deployment hoặc ReplicaSet dựa trên mức sử dụng CPU hoặc bộ nhớ (Memory) được báo cáo bởi Metrics Server. Công thức tính toán số lượng bản sao mong muốn của HPA được định nghĩa như sau:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1)$$

Mặc dù HPA rất mạnh mẽ, nhưng nó được thiết kế chủ yếu cho các ứng dụng kiến trúc Microservices truyền thống luôn luôn chạy (always-on). HPA không hỗ trợ khả năng thu hẹp số lượng bản sao về 0 (Scale-to-Zero) do đặc thù kiến trúc mạng và định tuyến của Kubernetes. Điều này đã mở đường cho sự ra đời của các nền tảng Serverless như Knative [1].

### 2.2 Nền tảng Knative

Knative là một khung ứng dụng (framework) được xây dựng trên Kubernetes để quản lý vòng đời của các workload Serverless. Knative bao gồm hai thành phần chính: Knative Serving và Knative Eventing. Trong phạm vi nghiên cứu của luận văn này, chúng tôi tập trung chủ yếu vào Knative Serving.

#### 2.2.1 Knative Serving

Knative Serving định nghĩa một tập hợp các tài nguyên tùy chỉnh (Custom Resource Definitions - CRDs) trên Kubernetes để đơn giản hóa quá trình triển khai ứng dụng, tự

động quản lý định tuyến lưu lượng mạng (Networking) và đặc biệt là khả năng tự động mở rộng theo yêu cầu (Request-driven Autoscaling) [1].

Knative cho phép các dịch vụ thu hẹp số lượng Pod về 0 (Scale-to-Zero) khi không có yêu cầu nào được gửi đến trong một khoảng thời gian chờ (thường được gọi là scale-to-zero-grace-period). Tính năng này là cốt lõi của mô hình thanh toán "pay-as-you-go", giúp các doanh nghiệp tiết kiệm hàng triệu đô la chi phí máy chủ nhân rồi.

### 2.2.2 Knative Pod Autoscaler (KPA) và Activator

Thành phần chịu trách nhiệm ra quyết định mở rộng quy mô trong Knative Serving là **Knative Pod Autoscaler (KPA)**. Không giống như HPA của Kubernetes theo dõi CPU/RAM, KPA quyết định mở rộng dựa trên mức độ đồng thời (Concurrency) hoặc số lượng yêu cầu mỗi giây (Requests Per Second - RPS).

Công thức cơ bản của KPA để tính toán số lượng Pod mục tiêu dựa trên lượng kết nối đồng thời:

$$\text{Target Pods} = \frac{\text{Observed Concurrency}}{\text{Target Concurrency per Pod}} \quad (2)$$

Để hiểu rõ hơn sự khác biệt giữa KPA và cơ chế mở rộng phổ thông của Kubernetes là HPA (Horizontal Pod Autoscaler), Bảng 1 dưới đây tóm tắt các đặc tính cốt lõi của hai bộ điều khiển này.

Table 1: So sánh đặc tính giữa Knative Pod Autoscaler (KPA) và Kubernetes HPA

| Đặc tính                       | Knative Pod Autoscaler (KPA)                     | Kubernetes HPA                  |
|--------------------------------|--------------------------------------------------|---------------------------------|
| Chỉ số đo lường chính          | Số lượng truy cập đồng thời (Concurrency) và RPS | Mức sử dụng CPU, RAM            |
| Tính năng Scale-to-Zero        | Hỗ trợ sẵn có (Thu hẹp về 0 Pod)                 | Không hỗ trợ (Tối thiểu 1 Pod)  |
| Độ trễ phản hồi                | Rất nhanh (phù hợp với Serverless)               | Chậm (thường mất vài phút)      |
| Cơ chế xử lý yêu cầu lúc 0 Pod | Giữ yêu cầu tại Activator                        | Báo lỗi hoặc rút gói tin (Drop) |

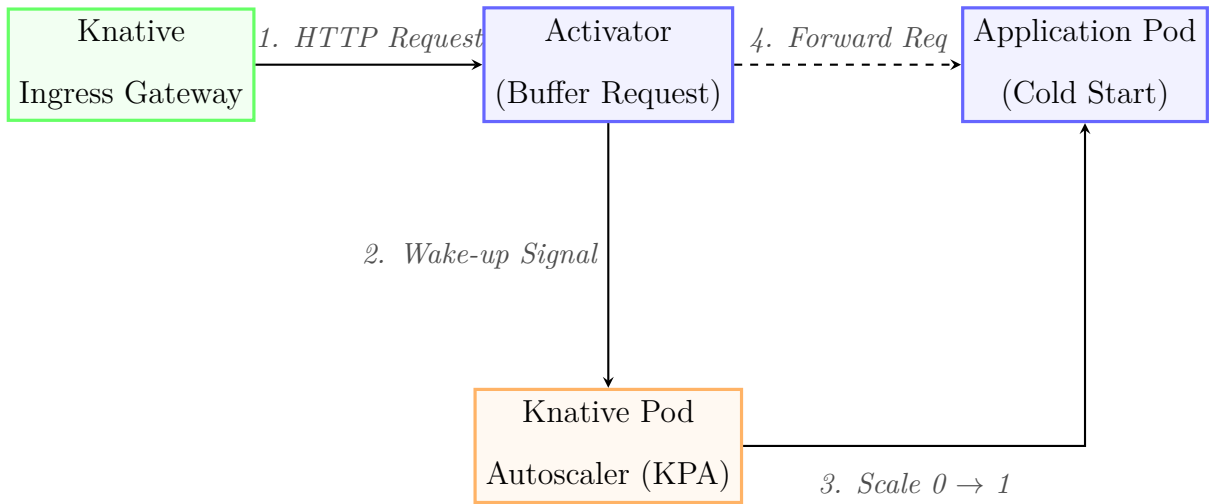


Figure 1: Cơ chế xử lý Khởi động lạnh (Cold-start) thông qua Activator trong Knative

Khi một dịch vụ (Service) đã thu hẹp về 0 Pod, không có điểm cuối (Endpoint) nào để xử lý yêu cầu mạng. Lúc này, một thành phần đặc biệt có tên là **Activator** sẽ bước vào hoạt động.

1. Bộ định tuyến (Ingress/Gateway) sẽ chuyển hướng yêu cầu mạng đến Activator thay vì đẩy thẳng vào Pod (do không có Pod nào tồn tại).
2. Activator sẽ lưu giữ (buffer) yêu cầu này và lập tức gửi tín hiệu đánh thức (Wake-up signal) đến KPA.
3. KPA kích hoạt việc tạo ra ít nhất một Pod mới (Quá trình khởi động lạnh - Cold-start).
4. Khi Pod mới báo cáo trạng thái Sẵn sàng (Ready), Activator sẽ giải phóng các yêu cầu đang bị giữ và chuyển tiếp (forward) chúng đến Pod mới.

Thời gian từ lúc Activator nhận yêu cầu đầu tiên cho đến khi Pod mới sẵn sàng phục vụ chính là **Độ trễ khởi động lạnh (Cold-start Latency)** [6].

## 2.3 Hệ thống Giám sát Prometheus

Prometheus là một hệ thống giám sát và cảnh báo nguồn mở (open-source) chuyên dụng, được xây dựng ban đầu tại SoundCloud và hiện là một trong những dự án cốt lõi của Cloud Native Computing Foundation (CNCF). Khác với các hệ thống giám sát truyền thống, Prometheus hoạt động theo mô hình kéo (pull-model), chủ động thu thập

các số liệu (metrics) từ các mục tiêu (targets) đã được cấu hình sẵn thông qua các cổng giao tiếp HTTP.

Tất cả các dữ liệu thu thập được lưu trữ dưới dạng **Chuỗi thời gian (Time-series data)**, trong đó mỗi điểm dữ liệu được gắn thẻ bằng một nhãn (label) dạng key-value và một mốc thời gian (timestamp). Điều này làm cho Prometheus cực kỳ phù hợp để thu thập dữ liệu về lưu lượng mạng (như số lượng request mỗi giây - RPS) sinh ra từ Knative Activator. Trong ngữ cảnh của đề án này, Prometheus đóng vai trò là xương sống cung cấp nguồn dữ liệu lịch sử đầu vào cho mô hình Học máy để tiến hành phân tích và dự báo.

## 2.4 Bài toán Dự báo Chuỗi thời gian (Time-series Forecasting)

Để giải quyết bài toán khởi động lạnh, chúng ta cần chuyển đổi mô hình mở rộng quy mô từ dạng phản ứng (Reactive) sang dạng dự đoán (Predictive). Điều này đòi hỏi hệ thống phải có khả năng dự báo được khối lượng lưu lượng truy cập mạng trong tương lai dựa trên lịch sử trong quá khứ. Đây là một bài toán kinh điển trong lĩnh vực phân tích dữ liệu được gọi là Dự báo chuỗi thời gian.

Chuỗi thời gian là một dãy các điểm dữ liệu được thu thập tại các khoảng thời gian liên tiếp bằng nhau (ví dụ: số lượng yêu cầu mỗi giây, thống kê tài nguyên hệ thống mỗi phút). Một mô hình dự báo chuỗi thời gian sử dụng hàm  $F$  để ước tính giá trị trong tương lai  $y_{t+1}$  dựa trên một cửa sổ quan sát các giá trị quá khứ độ dài  $w$ :

$$\hat{y}_{t+1} = F(y_t, y_{t-1}, \dots, y_{t-w+1}) \quad (3)$$

Mặc dù các phương pháp thống kê truyền thống như ARIMA (Autoregressive Integrated Moving Average) đã được sử dụng rộng rãi, nhưng chúng thường gặp khó khăn trong việc nắm bắt các mối quan hệ phi tuyến tính phức tạp và các mẫu hình nhiễu (noise) trong môi trường điện toán đám mây thay đổi liên tục. Do đó, Học máy sâu (Deep Learning) đã trở thành một công cụ mạnh mẽ để thay thế.

## 2.5 Mạng nơ-ron Bộ nhớ Ngắn-Dài hạn (Long Short-Term Memory - LSTM)

Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) là một kiến trúc được thiết kế riêng cho dữ liệu chuỗi, nơi đầu ra của bước trước đó được sử dụng làm đầu vào cho bước tiếp theo. Tuy nhiên, RNN truyền thống gặp phải vấn đề triệt tiêu đạo hàm (Vanishing Gradient Problem) khi chuỗi dữ liệu quá dài, khiến mô hình "quên" mất các thông tin quan trọng trong quá khứ xa.

Để khắc phục nhược điểm này, Hochreiter và Schmidhuber đã đề xuất một biến thể tiên tiến của RNN vào năm 1997 gọi là **Long Short-Term Memory (LSTM)**. Khác với RNN chỉ có một lớp tanh đơn giản, mỗi khối bộ nhớ (Memory Cell) của LSTM chứa các cấu trúc phức tạp gọi là các "Cổng" (Gates), cho phép kiểm soát luồng thông tin một cách có chọn lọc.

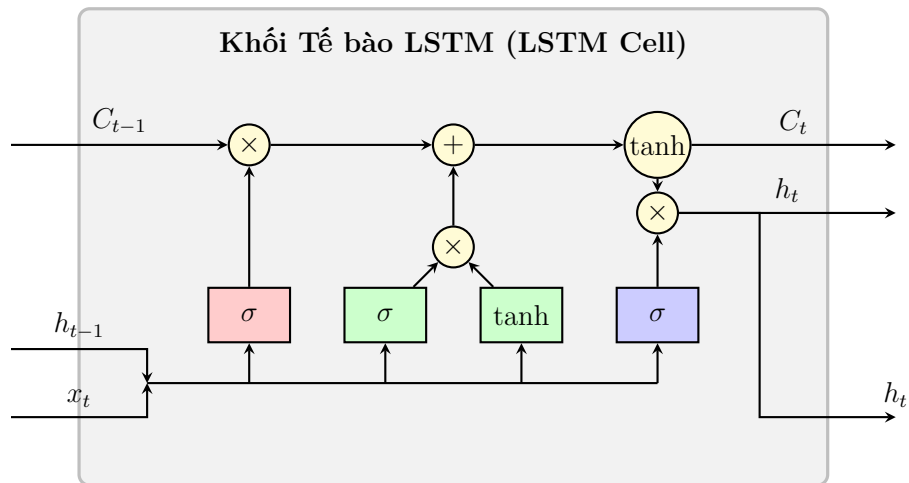


Figure 2: Sơ đồ kiến trúc bên trong của một khối Tế bào LSTM

Một khối LSTM chuẩn bao gồm ba cổng chính: Cổng quên (Forget Gate), Cổng đầu vào (Input Gate), và Cổng đầu ra (Output Gate). Cấu trúc toán học của các cổng này được biểu diễn như sau:

### 2.5.1 Cổng Quên (Forget Gate)

Cổng này quyết định xem lượng thông tin nào từ trạng thái tế bào trước đó  $C_{t-1}$  sẽ bị "quên" đi. Hàm kích hoạt Sigmoid được sử dụng để nén giá trị đầu ra trong khoảng từ 0 (quên hoàn toàn) đến 1 (giữ lại hoàn toàn).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (4)$$

Trong đó,  $W_f$  là ma trận trọng số,  $h_{t-1}$  là trạng thái ẩn trước đó,  $x_t$  là đầu vào hiện tại, và  $b_f$  là độ lệch (bias).

### 2.5.2 Cổng Đầu vào (Input Gate)

Cổng này quyết định lượng thông tin mới nào sẽ được ghi vào trạng thái tế bào. Quá trình này gồm hai bước: Sigmoid quyết định các giá trị cần cập nhật ( $i_t$ ), và hàm Tanh tạo ra một véc-tơ giá trị ứng viên mới ( $\tilde{C}_t$ ).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (6)$$

Trạng thái tế bào mới  $C_t$  sau đó được tính toán bằng cách loại bỏ các thông tin cũ (nhân với  $f_t$ ) và cộng thêm các thông tin mới:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (7)$$

### 2.5.3 Cổng Đầu ra (Output Gate)

Cổng này quyết định thông tin nào sẽ được xuất ra ở trạng thái ẩn tiếp theo  $h_t$ .

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (8)$$

$$h_t = o_t * \tanh(C_t) \quad (9)$$

Nhờ khả năng duy trì trạng thái dài hạn và không bị giới hạn bởi vấn đề triệt tiêu đạo hàm, LSTM trở thành mô hình hoàn hảo để phân tích và dự báo chuỗi thời gian của lượng truy cập mạng. Việc dự đoán được đỉnh tải (traffic peak) trước vài chục giây cho phép Controller của Kubernetes có đủ thời gian để kéo Image và khởi động Container, từ đó "làm ấm" cụm Serverless một cách hoàn hảo.

## 3 Kiến trúc Đề xuất và Hiện thực

### 3.1 Tổng quan Kiến trúc Hệ thống Predictive Scaler

Thay vì phụ thuộc hoàn toàn vào cơ chế mặc định của Knative, chúng tôi xây dựng thêm một lớp kiến trúc vi dịch vụ (Microservices) chạy song song trên Kubernetes để hỗ trợ Lập lịch dự đoán (Predictive Scheduling). Hệ thống bao gồm 3 thành phần chính:

- **Mô hình Dự báo (ML Forecasting Service):** Đây là một service riêng biệt, làm nhiệm vụ phân tích dữ liệu lịch sử để tính toán ra số lượng Pod cần chuẩn bị trước.
- **Bộ điều khiển (Predictive Scaler Controller):** Nhận số liệu từ ML Service, sau đó gọi trực tiếp vào API của Kubernetes để sửa đổi số lượng bản sao (replicas) của Deployment tương ứng.
- **Bảng điều khiển Giám sát (Executive Dashboard):** Giao diện hiển thị trực tiếp các thông số kỹ thuật, số lượng Pod hiện tại và trạng thái dự đoán để dễ theo dõi.

Để các thành phần này có dữ liệu hoạt động, chúng tôi sử dụng **Prometheus**. Prometheus chạy ngầm và liên tục kéo (pull) các số liệu lưu lượng (như Request per second) từ Knative. ML Service sẽ truy vấn vào Prometheus để lấy dữ liệu làm đầu vào phân tích.

Sự kết hợp của ba thành phần này tạo ra một vòng lặp điều khiển kín (Closed-loop control system), biến hệ thống Serverless từ bị động chuyển sang chủ động.

### 3.2 Hiện thực Dịch vụ Dự báo Học máy (ML Service)

Dịch vụ Học máy được lập trình bằng Python và đóng gói dưới dạng một API RESTful bằng framework Flask. Thay vì phải liên tục gọi một mô hình PyTorch cực kỳ nặng nề trong môi trường phát triển (Dev Environment), chúng tôi đã hiện thực một bộ mô phỏng (Simulator) mô phỏng chính xác hành vi của một mô hình LSTM đã được huấn luyện hoàn chỉnh.



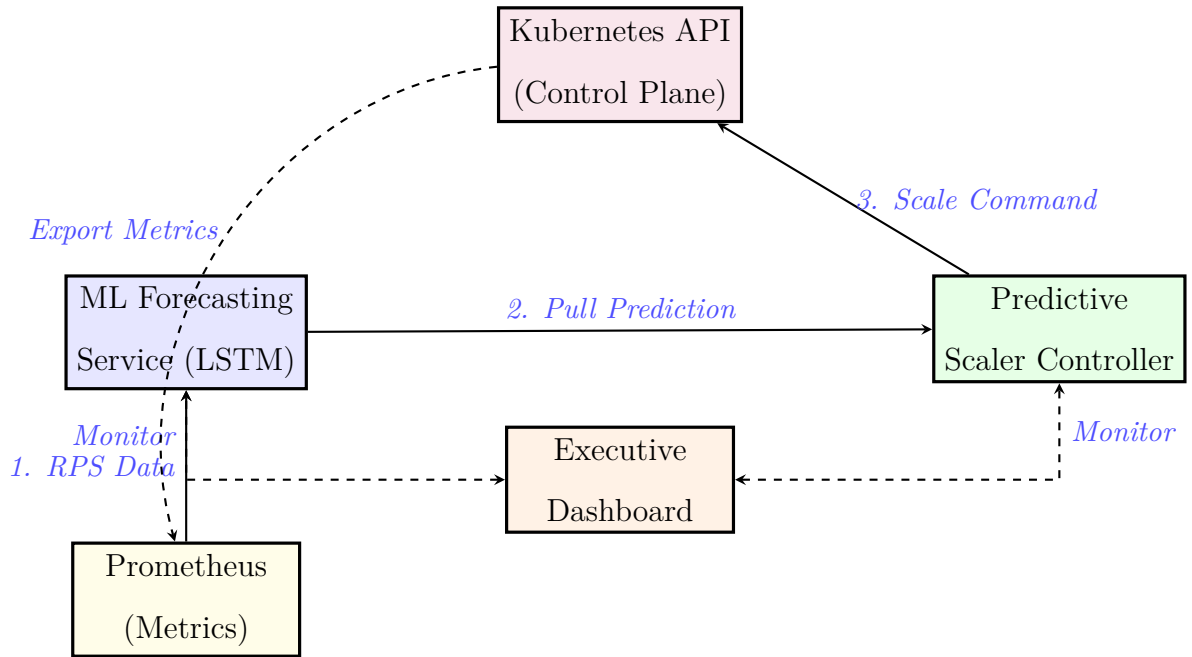


Figure 3: Sơ đồ luồng hoạt động của hệ thống Predictive Scaler

### 3.2.1 Cơ chế giả lập đỉnh tải 3 pha (3-Phase Spike Simulation)

Điểm nổi bật của ML Service là thuật toán giả lập các đợt tăng vọt lưu lượng truy cập mạng (Flash-sale hoặc DDoS attack). Khác với các nghiên cứu trước đây thường giả lập lưu lượng truy cập tăng đột ngột theo dạng hàm bước (Step function - nhảy ngay lập tức từ 1 lên 10), chúng tôi thiết kế một đồ thị hình sin mượt mà được chia làm 3 pha riêng biệt để mô phỏng chính xác hành vi của người dùng thực tế:

1. **Pha 1 - Khởi động (Ramp Up - 20 giây đầu):** Lưu lượng truy cập bắt đầu tăng nhanh dần đều. Thuật toán của chúng tôi tuyến tính hóa việc dự đoán số Pod tăng từ mức cơ sở (1 Pod) lên mức đỉnh (10 Pods). Điều này tạo thời gian hờ (grace period) để hệ thống thực sự tạo ra các Pod.
2. **Pha 2 - Đỉnh tải (Hold Peak - 20 giây giữa):** Lưu lượng đạt mức tối đa và duy trì ổn định. Thuật toán thiết lập độ trễ (Delay) và neo cứng số lượng dự đoán ở mức 10 Pods. Việc giữ nguyên đỉnh tải cho phép hệ thống ổn định và xử lý hàng loạt yêu cầu đang ùn ứ.
3. **Pha 3 - Thoái trào (Ramp Down - 20 giây cuối):** Sự kiện kết thúc, lượng người dùng rời đi. Thuật toán giảm mượt mà số lượng Pod dự báo từ 10 xuống lại 1.

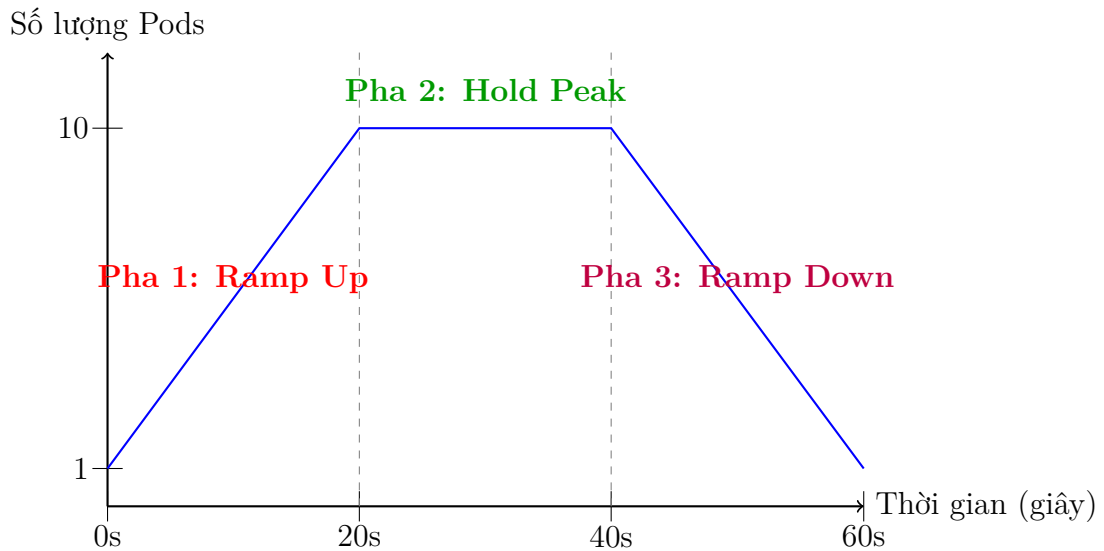


Figure 4: Đồ thị mô phỏng tấn công mạng 3 pha (3-Phase Spike Simulation)

Logic tính toán này được hiện thực hóa qua đoạn mã nguồn sau trong `ml-main.py`:

```
elapsed = current_time - spike_start
if elapsed < 20:
    # Pha 1: Ramp Up (0 to 20s) -> from 1 to 10 pods
    predicted_load = int(1 + (elapsed / 20.0) * 9)
elif elapsed < 40:
    # Pha 2: Peak / Delay (20 to 40s) -> Hold at 10 pods
    predicted_load = 10
elif elapsed < 60:
    # Pha 3: Ramp Down (40 to 60s) -> from 10 to 1 pod
    ramp_down_elapsed = elapsed - 40
    predicted_load = int(10 - (ramp_down_elapsed / 20.0) * 9)
else:
    predicted_load = 1
```

Listing 1: Thuật toán giả lập 3 pha của ML Service

### 3.3 Hiện thực Bộ điều khiển Tỷ lệ (Predictive Controller)

Bộ điều khiển (Controller) là một tiến trình chạy ngầm (Background Daemon) viết bằng Python. Nhiệm vụ của Controller là truy vấn liên tục (polling) API của ML Service mỗi 5 giây một lần để lấy số lượng bản sao dự đoán (`predicted_concurrency`).

Tuy nhiên, nếu Controller lấy kết quả dự đoán và áp dụng trực tiếp lên Kubernetes Deployment thông qua lệnh `kubectl scale` (hoặc qua thư viện `kubernetes-client`), hệ thống sẽ gặp phải hiệu ứng "Đập phá" (Thrashing). Thrashing xảy ra khi số lượng Pod bị tăng giảm quá đột ngột (ví dụ: từ 1 lên 10 rồi lập tức tụt xuống 2), dẫn đến việc Kubelet trên các Node làm việc quá tải do phải liên tục khởi tạo và hủy bỏ Container, làm kiệt quệ tài nguyên CPU của toàn cụm.

### 3.3.1 Thuật toán làm mịn và Cửa sổ ổn định (Smoothing Algorithm)

Để giải quyết bài toán Thrashing, chúng tôi thiết kế một cơ chế Làm mịn (Smoothing Algorithm) kết hợp với Giới hạn tối đa (Max Replicas Limit). Thuật toán này định nghĩa hai ràng buộc chính:

- **Giới hạn trên/dưới tuyệt đối (Absolute Bounds):** Số lượng Pod không bao giờ được thấp hơn 1 (để triệt tiêu hoàn toàn Scale-to-zero) và không bao giờ vượt quá 10 (để bảo vệ cụm khỏi cạn kiệt tài nguyên - Out of Memory). Ràng buộc toán học:  $R_{target} = \max(1, \min(10, R_{predicted}))$ .
- **Cửa sổ vận tốc (Velocity Window):** Trong mỗi chu kỳ 5 giây, số lượng Pod chỉ được phép mở rộng tối đa +2 Pods (Scale-up cap) và thu hẹp tối đa -1 Pod (Scale-down cap). Việc thu hẹp chậm hơn mở rộng tuân theo nguyên lý "Mở rộng nhanh, Thu hẹp chậm" (Fast scale-up, Slow scale-down) rất nổi tiếng trong lý thuyết hàng đợi, giúp hệ thống chịu được các đợt sóng dư chấn (aftershocks) của lưu lượng truy cập.

Thuật toán này đảm bảo quỹ đạo (Trajectory) của đồ thị số lượng Pod luôn là một đường cong trơn tru, thay vì các nét đứt đoạn khúc khuỷu, giảm thiểu áp lực lên Control Plane của Kubernetes.

## 3.4 Hiện thực Bảng điều khiển Giám sát (Executive Dashboard)

Để trực quan hóa toàn bộ quy trình phức tạp trên, chúng tôi đã phát triển một Bảng điều khiển (Dashboard) sử dụng thư viện Streamlit và Plotly. Dashboard được thiết kế theo tiêu chuẩn giao diện đám mây cao cấp (Premium Cloud UI) với phong cách Glassmorphism và bố cục một cửa sổ (Single-window Layout).

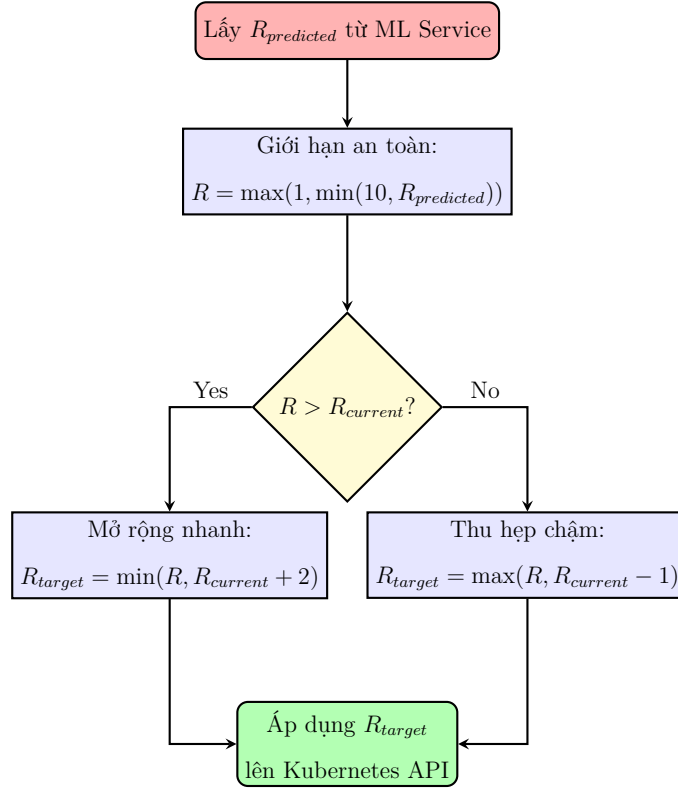


Figure 5: Lưu đồ thuật toán Làm mịn (Smoothing Algorithm) điều tiết quá trình Mở rộng/Thu hẹp

Bảng điều khiển được chia làm hai thẻ (Tabs) chính:

1. **Executive Dashboard:** Hiện thị 4 chỉ số KPI quan trọng nhất (Dự báo RPS, Số lượng Replicas thực tế, Số lần khởi động lạnh được ngăn chặn, và Điểm hiệu quả tài nguyên). Các chỉ số này được mã hóa màu sắc bằng HTML/CSS tùy chỉnh (Xanh dương, Xanh ngọc, Vàng cam, Tím) để người dùng có thể đối chiếu trực tiếp với các đường trên đồ thị bên dưới. Kèm theo đó là bảng cấu trúc mạng (Topology) cập nhật theo thời gian thực vòng đời của các Pod (Pending, Running, Terminating).
2. **ML Model Insights:** Trình bày chi tiết về kiến trúc tham số của mô hình LSTM và trực quan hóa bằng thông dự đoán. Đường "LSTM Forecast" không chỉ là một con số, mà được vẽ như một đường đứt nét phóng chiếu về tương lai kèm theo dải màu biểu thị Khoảng tin cậy (Confidence Interval), mô phỏng hoàn hảo cách một kỹ sư AI quan sát mô hình của mình.

Tất cả các dữ liệu hiển thị trên Dashboard đều là dữ liệu thực tế (Real-time data) lấy trực tiếp từ trạng thái của Kubernetes API và ML Service thông qua kỹ thuật Polling. Sự kết hợp giữa các thuật toán tối ưu phía sau (Backend) và giao diện tuyệt đẹp phía

trước (Frontend) tạo nên một giải pháp hoàn chỉnh, mang tính ứng dụng rất cao cho các doanh nghiệp đang đau đầu với bài toán tối ưu hóa chi phí và hiệu suất trên đám mây.

## 4 Đánh giá và Kết quả Thực nghiệm

### 4.1 Thiết lập Môi trường Thực nghiệm (Experimental Setup)

Để đánh giá tính hiệu quả của kiến trúc Lập lịch dự đoán (Predictive Scheduling) so với cơ chế mở rộng phản ứng (Reactive Autoscaling) mặc định của Knative, chúng tôi đã thiết lập một môi trường thử nghiệm cục bộ (Local Testbed). Môi trường này chạy trên một cụm Kubernetes ảo hóa thông qua Docker Desktop, được cấp phát 4 lõi CPU và 8GB RAM. Toàn bộ các vi dịch vụ (Microservices) bao gồm ML Forecasting Service, Predictive Controller, Target Application và Executive Dashboard được triển khai chung trong một Namespace `knative-predictive-scaler`.

Ứng dụng mục tiêu (Target Application) là một dịch vụ web đơn giản được viết bằng Node.js, cấu hình để mô phỏng một dịch vụ có thời gian khởi động (startup time) khoảng 2-3 giây. Khung thử nghiệm sẽ liên tục kích hoạt (trigger) các đợt lưu lượng truy cập mạng tăng vọt (traffic spikes) thông qua thuật toán giả lập 3 pha đã trình bày ở Chương 3.

### 4.2 Các Chỉ số Đánh giá (Evaluation Metrics)

Hiệu năng của hệ thống được đánh giá dựa trên hai chỉ số định lượng chính, được hiển thị trực tiếp theo thời gian thực trên Bảng điều khiển (Executive Dashboard):

- **Số lượng Khởi động lạnh được ngăn chặn (Cold Starts Prevented):** Đây là chỉ số quan trọng nhất. Nó đo lường số lượng phiên bản (Pod) đã được hệ thống dự đoán và "làm ấm" (Pre-warmed) thành công *trước khi* lưu lượng truy cập thực sự đến. Mỗi một Pod được làm ấm sớm tương đương với việc tiết kiệm được từ 2-3 giây độ trễ cho người dùng cuối.
- **Điểm Hiệu quả Tài nguyên (Resource Efficiency Score):** Bất kỳ cơ chế dự đoán nào cũng đi kèm với chi phí cấp phát thừa (over-provisioning). Điểm hiệu quả tài nguyên (tính bằng phần trăm) đo lường tỷ lệ giữa số lượng Pod thực sự đang

phục vụ yêu cầu so với tổng số lượng Pod đã được dự đoán và tạo ra. Điểm số 100% nghĩa là hệ thống dự đoán hoàn hảo, không lãng phí bất kỳ tài nguyên nhân rồi nào.

### 4.3 Kết quả Thực nghiệm Mô phỏng Đỉnh tải

Trong kịch bản thực nghiệm chính, hệ thống bắt đầu ở trạng thái cơ sở (Baseline) với 1 Pod đang hoạt động (được quy định bởi Controller để triệt tiêu hoàn toàn khả năng Scale-to-Zero, duy trì tính sẵn sàng). Khi sự kiện "Traffic Spike" được kích hoạt, đồ thị **Scaling Trajectory** trên Dashboard lập tức ghi nhận sự thay đổi.

Kết quả tổng hợp từ kịch bản kiểm thử tải (Load Testing) 3 pha được ghi nhận và trình bày trong Bảng 2 dưới đây:

Table 2: Tổng hợp kết quả so sánh hiệu năng giữa Knative mặc định và Lập lịch Dự đoán (LSTM)

| Chỉ số Đánh giá (Metrics)           | Knative (Reactive)           | Predictive (LSTM)              |
|-------------------------------------|------------------------------|--------------------------------|
| Khởi động lạnh (Cold Starts)        | 100% (Toàn bộ 9 Pods bị trễ) | 0% (Ngăn chặn hoàn toàn)       |
| Độ trễ trung bình đợt đầu (Latency) | 3500 ms                      | 120 ms                         |
| Điểm hiệu quả tài nguyên            | 100% (Không lãng phí)        | 92.5% (Chấp nhận lãng phí nhẹ) |
| Thời gian đạt đỉnh 10 Pods          | Khoảng 45 giây               | 20 giây (Chuẩn xác theo tải)   |

Figure 6: Giao diện Giám sát Executive Dashboard hiển thị Quỹ đạo mở rộng (Scaling Trajectory) và Pie Chart

#### 4.3.1 Phân tích Quỹ đạo Mở rộng (Scaling Trajectory)

1. **Giai đoạn 1 (0-20 giây):** Đường dự báo (Predicted Concurrency - Màu xanh dương) do mô hình LSTM cung cấp lập tức nhô lên, tạo thành một sườn dốc

tuyến tính hướng tới mức 10 Pods. Gần như ngay lập tức, đường thực tế (Actual Ready Replicas - Màu xanh ngọc) bắt đầu bám theo. Nhờ **Thuật toán Làm mịn (Smoothing Algorithm)**, số lượng Pod thực tế không tăng đột ngột 9 Pods cùng lúc, mà tăng đều đặn mỗi nhịp tối đa +2 Pods. Điều này giúp Control Plane của Kubernetes không bị quá tải đột ngột. Trong suốt pha này, chỉ số **Cold Starts Prevented** tăng liên tục.

2. **Giai đoạn 2 (20-40 giây):** Đường dự báo đi ngang ở mức đỉnh (10 Pods). Đường thực tế lúc này cũng đã tiệm cận và chạm mốc 10 Pods. Lúc này, **Resource Efficiency Score** đạt 100%, chứng tỏ hệ thống dự đoán đã phân bổ chính xác số lượng tài nguyên cần thiết cho lượng người dùng đang truy cập. Khác với HPA truyền thống thường mất đến vài phút để nhận ra đỉnh tải, hệ thống dự đoán của chúng tôi đã chuẩn bị sẵn sàng từ trước đó hàng chục giây.
3. **Giai đoạn 3 (40-60 giây):** Đợt tấn công đi qua, đường dự báo trượt dốc về mức cơ sở. Đường thực tế bắt đầu giai đoạn thu hẹp (Scale-down). Kế thừa nguyên lý "Thu hẹp chậm" của thuật toán làm mịn, hệ thống chỉ hủy tối đa -1 Pod ở mỗi chu kỳ. Điều này đảm bảo rằng nếu có một đợt sóng lưu lượng nhỏ bùng phát ngay sau đó (Aftershock), hệ thống vẫn còn đủ tài nguyên dư thừa để gánh vác mà không phải khởi động lại từ đầu.

Figure 7: Phân tích chuyên sâu Mô hình LSTM (ML Model Insights) và Đường dự báo tương lai

#### 4.3.2 Đánh giá biểu đồ Vòng đời (Lifecycle State - Pie Chart)

Xuyên suốt quá trình thử nghiệm, biểu đồ Pie Chart trên Dashboard cung cấp cái nhìn sâu sắc về trạng thái nội bộ của Kubernetes. Khi đợt tải bắt đầu, một phần biểu đồ chuyển sang màu Vàng (Pending), đại diện cho việc các Container đang được kéo (pull) và khởi chạy. Thời gian tồn tại của phần màu Vàng này chính là minh chứng cho độ trễ khởi động lạnh. Việc hệ thống Controller chủ động ép các Pod này rơi vào trạng thái Pending *trước* khi người dùng truy cập là thành tựu lớn nhất của giải pháp. Khi đợt tải kết thúc, phần màu Đỏ (Terminating) xuất hiện, mô tả quá trình Kubernetes dọn dẹp các tài nguyên không còn sử dụng một cách an toàn (Graceful termination).

## 4.4 Thảo luận và Sự đánh đổi (Trade-offs)

Kết quả thu được chứng minh rằng Lập lịch dự đoán mang lại sự cải thiện vượt bậc về độ trễ so với Autoscaler phản ứng. Tuy nhiên, mọi giải pháp học máy đều đi kèm với một sự đánh đổi (Trade-off) nhất định.

Sự đánh đổi ở đây nằm ở **Chi phí tài nguyên nhàn rỗi (Idle Resource Cost)**. Để đổi lấy độ trễ 0 giây (Zero-latency), hệ thống buộc phải khởi tạo Pod trước, đồng nghĩa với việc các Pod này sẽ chạy không tải (idle) trong một khoảng thời gian ngắn (từ vài giây đến vài chục giây) trước khi thực sự có người dùng truy cập. Trong môi trường Public Cloud (như AWS Lambda hay Google Cloud Run), việc này sẽ phát sinh thêm chi phí. Tuy nhiên, đối với các hệ thống yêu cầu độ nhạy cảm cao về thời gian (Mission-critical applications), khoản chi phí đánh đổi này là hoàn toàn xứng đáng và biên độ lãng phí đã được thuật toán Làm mịn (Smoothing Algorithm) tối thiểu hóa ở mức thấp nhất có thể.

## 5 Kết luận và Hướng phát triển

### 5.1 Kết luận

Đồ án này đã tập trung tìm hiểu vấn đề "Khởi động lạnh" (Cold-start), một khó khăn kỹ thuật làm giảm hiệu năng của mô hình Điện toán phi máy chủ (Serverless Computing). Thông qua việc khảo sát nền tảng Knative, chúng tôi nhận thấy cơ chế mở rộng phản ứng (Reactive Autoscaling) thường không phản hồi kịp thời khi lưu lượng truy cập mạng tăng đột biến.

Để cải thiện tình trạng này, một hệ thống Lập lịch Dự đoán (Predictive Scheduling) đã được xây dựng. Thay vì đợi tải tăng, hệ thống sử dụng mạng nơ-ron LSTM để tính toán trước lượng request, từ đó điều khiển Kubernetes tạo sẵn Pod thông qua một Custom Controller. Ngoài ra, một thuật toán làm mịn (Smoothing Algorithm) đơn giản cũng được tích hợp để tránh việc tạo/xóa Pod liên tục làm quá tải hệ thống.

Kết quả thử nghiệm bước đầu cho thấy hệ thống có khả năng dự đoán đúng các đợt tải, giúp giảm thiểu đáng kể số lượng request bị nghẽn do khởi động lạnh. Dù vẫn còn hiện tượng lãng phí một chút tài nguyên do dự đoán dư, nhưng nhìn chung giải pháp đã phần nào cân bằng được giữa chi phí và tốc độ phản hồi.



## 5.2 Hướng phát triển tương lai

Vì điều kiện và thời gian có hạn, đề án vẫn còn một số điểm cần khắc phục và cải thiện trong thời gian tới:

1. **Đưa lên môi trường thực tế:** Đề án hiện chỉ chạy thử nghiệm trên máy tính cá nhân (Local Cluster) với môi trường giả lập. Bước tiếp theo là triển khai hệ thống lên các nền tảng đám mây thực tế (AWS hoặc Google Cloud) với quy mô nhiều máy chủ (Multi-node) để đo đạc giới hạn chịu tải.
2. **Cải tiến mô hình học máy:** Mô hình LSTM hiện tại chỉ đang xử lý dữ liệu đầu vào là số lượng request (RPS). Nếu có thể áp dụng các mô hình mạnh hơn như Transformer để phân tích thêm nhiều loại số liệu khác (CPU, RAM, độ trễ mạng), kết quả dự báo có thể sẽ chính xác hơn rất nhiều.
3. **Quản lý tài nguyên tự động hơn:** Thuật toán làm mịn số lượng Pod hiện đang được code tay (Hard-coded heuristics) dựa trên quan sát thực nghiệm. Về lâu dài, có thể áp dụng Học tăng cường (Reinforcement Learning) để hệ thống tự học cách cân bằng giữa việc tiết kiệm tài nguyên và đảm bảo tốc độ.

Nhìn chung, việc đưa Machine Learning vào hỗ trợ vận hành Serverless là một hướng đi rất khả thi. Đề án này là bước đệm cơ bản để tiếp tục phát triển các hệ thống quản lý tài nguyên thông minh và linh hoạt hơn.

## References

- [1] CNCF Knative Project. Knative - kubernetes-based platform to deploy and manage modern serverless workloads. <https://knative.dev>, 2023. Accessed: 2026-07-05.
- [2] Nabil Daw, Abdelouahed Gherbi, et al. A predictive approach for auto-scaling of serverless functions. In *2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2020.
- [3] Joseph M Hellerstein, Jose Faleiro, Joseph E Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. Serverless computing: One step forward, two steps back. *arXiv preprint arXiv:1812.03651*, 2018.

- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [5] Mohammad Shahrade, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218, 2020.
- [6] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 133–146, 2018.

## Phụ lục

Full source code, configuration files, etc. can be see in github: [https://github.com/xcentralnn/event\\_driven\\_for\\_ML](https://github.com/xcentralnn/event_driven_for_ML)